CN & SCALER · SESSION 4

nginx

and the code that explains it

A trimmed history · what nginx actually changed · the features you configure · and one long walk through the source.

nginx 1.31.5 · github.com/nginx/nginx · companion repo: nginx-from-scratch

THE FOUR HOURS

Where we are going

01

History, compressed

Who actually wrote the first web server. Apache outrunning fork(). J2EE's answer. Why 2002 needed something new.

02

What nginx changed

It did not make the worker cheaper. It deleted the worker. Event loop, epoll, sendfile, reverse proxy.

03

The features you configure

Caching, routing precedence, upstreams and weights, fallbacks — each one next to the code that implements it.

04

The walkthrough

Ten stops in the nginx source. Then our own seven servers, from fork() to FastCGI.

Remember the evolution. You can forget the dates.

When my kids study chemistry they ask why they need to remember scientists' names and dates. The names matter less than the chronology does.

Understanding the evolution tells you what the limitation was, and what was proposed to fix it. Sometimes you look at the same limitation and fix it a completely different way.

Understanding history is how you get to create a different timeline.

Every design in this session was somebody's answer to a constraint that no longer exists. Knowing which constraint is how you tell a law from a habit.

First, a correction worth making

The story you often hear

“Berners-Lee wrote the first server. It was called Mosaic. Mosaic became Apache.”

What actually happened

- Berners-Lee's server was CERN httpd, on a NeXT, 1990.
- Mosaic (NCSA, 1993) was a browser — Andreessen's, not a server.
- Apache descends from NCSA HTTPd — a different NCSA program.

Three separate programs, two of them at NCSA, one of them a browser. The confusion is understandable — and getting it right is the difference between a story and a lineage.

The actual lineage

1990	**CERN httpd**	Berners-Lee, on a NeXT. The first web server.
1993	**NCSA Mosaic**	A browser. Inline images. The web goes mainstream.
1993	**NCSA HTTPd**	Rob McCool. A server — and it invents CGI.
1994	**McCool leaves**	Development stalls. Webmasters trade patches by mail.
1995	**Apache**	That patch set, continued as a project. “A patchy server.”
1996	**Apache #1**	Passes NCSA HTTPd. Stays #1 for twenty years.
1996–99	**IIS, then the boom**	Microsoft fights for single-digit share. lighttpd and friends arrive.

Apache spent a decade trying to outrun fork()

Apache 1.3

prefork

A pool of processes, forked at startup instead of per request. The setup cost moves off the request path. One process still parks on one connection.

Apache 2.0

worker MPM

Processes each running many threads. 8 MB of stack becomes 8 MB shared plus a stack each. 1999–2002: pthreads on Linux are new, buggy, and everyone is learning what reentrant means.

Apache 2.4

event MPM

A listener thread parks idle keep-alives on epoll and only hands a worker thread a connection when it has actual work. Default on Unix since 2.4.

Three architectures, one question: how do I make the per-connection worker cheaper? Every answer was a good answer. None of them was a different question.

Java's answer: stop creating things

```
// CGI, 1993
per request:   fork() + exec() + load interpreter
               + parse script + run + exit

// Servlet, 1997
at startup:    load class, instantiate once
per request:   service(req, res)    <- a method call

// the object outlives the request. that is the trick.
```

- J2EE (1999) made the multi-layer application server the default shape: web tier, EJB tier, database.
- A servlet is instantiated once and serves every request on a thread from a pool.
- Connection pools do the same thing for the database handshake.
- **Same move as prefork. Same move as FastCGI. Take the expensive setup off the request path and pay it once.**

Our primary goal was to avoid creating a new process. Everything else was consequence.

So here is 2002

10,000

concurrent connections on one box. Dan Kegel names it the C10K problem in 1999

8 MB

of stack per thread, the default on Linux. Ten thousand threads is 80 GB of address space.

1024

descriptors, hard-coded as FD_SETSIZE in a header. ulimit does not move it.

Hardware is getting fast enough to serve ten thousand people from one machine. The software cannot, and not for want of trying — the limits are a per-connection stack, a per-connection scheduler entry, and a 1983 API with a bitmap in it.

Everyone is optimising the worker. Somebody is about to delete it.

Igor Sysoev, Rambler, 2002

- Rambler was one of Russia's largest portals. Its Apache servers were falling over under connection volume, not request volume.
- He started writing nginx in 2002 to solve C10K specifically.
- First public release 4 October 2004 — chosen for the anniversary of Sputnik.
- 1.0.0 did not arrive until April 2011. Seven years of shipping before the version number said “ready”.

He did not set out to write a faster Apache.

He set out to serve ten thousand mostly-idle connections from one process, and let something else generate the dynamic content.

That second half is why nginx is a reverse proxy and not an application server.

Apache asked: how do I make the worker cheaper?

nginx asked: why is there a worker per connection at all?

A connection is not a thing that needs a thread. It is a thing that needs a few hundred bytes of state and a place in a list, until it has something to say. When it does, some CPU handles it, finishes, and moves on.

One process per core. One event loop each. Ten thousand connections, and none of them owns anything.

One process per core, and a master that never touches a connection

master
binds :80, reads config, forks, handles signals. Nothing else.

worker 0
event loop

worker 1
event loop

worker N
event loop

cache manager
evicts LRU

cache loader
warms index

```
$ ps -o pid,ppid,args -C nginx
2663     1  nginx: master process
2665  2663  nginx: worker process
2666  2663  nginx: worker process
2667  2663  nginx: cache manager process
```

The count does not move under load. Run the same command against 01-fork/forkd and watch it move. Same fork() syscall — the only difference is how often you call it.

ngx_process_cycle.c ngx_master_process_cycle() ~L74 · ngx_start_worker_processes() ~L358 · ngx_process.c ngx_spawn_process() ~L87

All of nginx, in six lines

```
timer = ngx_event_find_timer();
ngx_process_events(cycle, timer, flags);   /* epoll_wait() */
ngx_event_process_posted(cycle, &ngx_posted_accept_events);
ngx_shmtx_unlock(&ngx_accept_mutex);
ngx_event_expire_timers();
ngx_event_process_posted(cycle, &ngx_posted_events);
```

- Line 2 is where a worker spends 99.9% of its life: asleep.
- **The timer is an ARGUMENT to the sleep. nginx never polls for timeouts.**
- Handlers do not run inside the poll; they are posted and drained after.

Everything else in src/http/ is a callback reached from line 2 or line 6. Read these six lines until they are boring.

src/event/ngx_event.c ngx_process_events_and_timers() ~L195 · compare: 04-epoll/epolld.c

Load balancing between workers, in one line of arithmetic

```
ngx_accept_disabled = ngx_cycle->connection_n / 8 - ngx_cycle->free_connection_n;
```

Read it as English: once I am using more than seven eighths of my connection slots, stop competing for new connections and let my siblings take them.

No coordinator

No shared counter, no leader, no heartbeat. Each worker decides alone, from a number it already has.

accept_mutex is now off

Default off since 1.11.3. EPOLLEXCLUSIVE (Linux 4.5+) and SO_REUSEPORT let the kernel do the fan-out instead.

A userspace fix for a kernel problem

The thundering herd was real in 2004. The kernel fixed it. The workaround is now dead weight — and nginx removed it.

src/event/ngx_event_accept.c ~L139 · src/event/ngx_event.c ngx_process_events_and_timers() ~L224 (ngx_trylock_accept_mutex)

select is O(watched). epoll is O(active). That is the whole difference.

	select()	epoll()
Where the watch list lives	In your process. Rebuilt every call.	In the kernel. Edited with epoll_ctl().
Cost per iteration	O(n) to build + O(n) in kernel + O(n) to scan	O(number of ready events)
10,000 idle, 10 active	30,000 units of work to find 10 events	10
Descriptor ceiling	FD_SETSIZE = 1024, compile-time, in a header	ulimit -n
What ulimit -n does	Nothing. The limit is a bitmap width.	Everything.
Portable?	Everywhere since 1983	Linux only (kqueue on BSD)

`ngx_select_module.c  ngx_select_process_events() ~L222  vs  ngx_epoll_module.c  ngx_epoll_process_events() ~L784`

Open both in a split window. Same signature, same job, ~200 lines each. Everything else in this comparison is downstream of one decision: who owns the watch list.

The bug every event-loop author writes exactly once

```
c = event_list[i].data.ptr;

instance = (uintptr_t) c & 1;
c = (ngx_connection_t *) ((uintptr_t) c & (uintptr_t) ~1);

if (c->fd == -1 || rev->instance != instance) {
    /* the stale event from a file descriptor
       that was just closed in this iteration */
    continue;
}
```

- epoll_wait returns a BATCH.
- Handling event 1 can close the connection event 7 points at.
- A new client on the recycled fd gets event 7 delivered to it.

Structs are aligned, so the low bit of the pointer is always zero. nginx borrows it as a generation flag and flips it on reuse. Free bit, no extra memory, one branch.

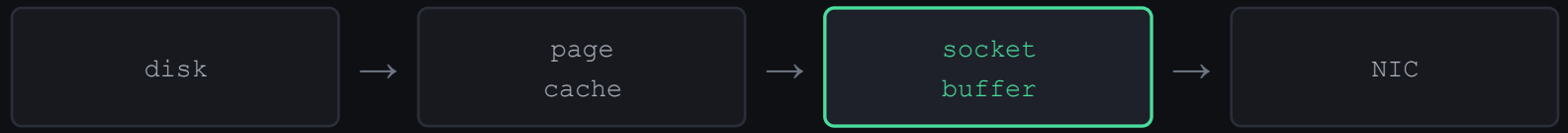
ngx_epoll_module.c ngx_epoll_process_events() ~L839 · 04-epoll/epolld.c does it with an explicit generation counter

sendfile: four copies become two

read() + write() — 4 copies, 2 mode switches per chunk



sendfile() — 2 copies, both DMA, 0 mode switches per chunk



The two boxes outlined above are the copies you pay CPU for. sendfile deletes the one through your address space — and with it, the two mode switches per 64 KiB chunk.

The data never enters your process.

Why you can't use it for everything

sendfile copies kernel-side. If you need to TOUCH the bytes — gzip, TLS, templating — they must come into user space. That is why `gzip on` and `sendfile on` fight, and why kTLS exists.

ngx_linux_sendfile_chain.c ngx_linux_sendfile_chain() ~L50 · ngx_linux_sendfile() ~L232 · 05-sendfile/

Now the honest part

```
$ ./sendfiled 8005 www/big.bin      # 64 MB, over loopback

mode=read      wall  37.4 ms | sys 37.3 |  2048 syscalls
mode=mmap      wall  23.3 ms | sys 23.1 |     4 syscalls
mode=sendfile  wall  33.2 ms | sys 32.3 |     1 syscall
```

On loopback, sendfile does not win on wall-clock time.

Students will run this and see it.

- There is no NIC and no DMA. The kernel memcpy's into the receiver's buffer either way, and curl is competing for the same CPU. The transfer is memory-bandwidth bound, so all three land in the same place.
- **What is real on any machine: 2048 syscalls become 1, and 64 MB never enters the process's address space.**
- The win shows up as headroom under concurrency — CPU you did not spend copying is CPU you spend on somebody else's request — not as a faster single transfer.

A benchmark that does not model the bottleneck measures the wrong thing. That lesson is worth as much as sendfile is.

Why “reverse” proxy? Because proxy was taken.

Forward proxy · Squid, 1996

Sits next to the CLIENTS.

One person on campus downloads an image; it is cached, and nobody else on campus downloads it again. It knows your users and it is protecting your uplink.

```
client → [proxy] → the whole internet
```

Reverse proxy · nginx

Sits next to the SERVERS.

Same caching. Plus: route by path or host, distribute across upstreams by weight or least-connections, fall back when one dies, terminate TLS, and hold ten thousand slow clients so your app never meets one.

```
the whole internet → [nginx] → your app servers
```

Same machinery, opposite side of the internet. The client does not configure a reverse proxy and usually cannot tell it is there — which is exactly why it can do things the client would never agree to.

And the killer feature nobody lists: it absorbs slow clients. Your app server never talks to a phone on 2G.

The cache: two structures, one of them on disk

```
proxy_cache_path temp/cache levels=1:2 keys_zone=demo:10m
                 max_size=100m inactive=60s use_temp_path=off;
```

`keys_zone=demo:10m`	10 MB of SHARED MEMORY holding the rbtree and the LRU queue. The index, not the data. Roughly 8,000 keys per megabyte.
`levels=1:2`	Two directory levels on disk, so no single directory holds a million files and destroys your filesystem's lookup time.
`max_size=100m`	Disk ceiling. The cache manager process evicts from the LRU tail to stay under it.
`inactive=60s`	Evicted if not READ for 60s — regardless of TTL. Different knob from proxy_cache_valid, and the one people confuse.

Ask the class: proxy_cache_valid is 10s and inactive is 60s. Which one wins, and what does each actually mean?

The LRU is one line

```
q   = ngx_queue_last(&cache->sh->queue);
fcn = ngx_queue_data(q, ngx_http_file_cache_node_t, queue);

wait = fcn->expire - now;
if (wait > 0) { ... break; }          /* tail is still fresh */

if (fcn->count == 0) {                /* nobody is streaming it */
    ngx_http_file_cache_delete(cache, q, name);
}
```

- **Take the tail. Is it old? Is nobody using it? Delete it.**
- That is the entire eviction policy.
- count == 0 is why an object being streamed to a client right now cannot be evicted out from under it.

src/core/ngx_queue.h is fifty lines of macros for an intrusive doubly-linked list. The cache LRU, the posted-events list and half of src/core/ ride on it. Read that file first.

ngx_http_file_cache.c ngx_http_file_cache_expire() ~L1860 · ..._lookup() ~L1029 · ..._open() ~L265

Two lines that save you at 3am

```
proxy_cache_lock on;
```

The anti-stampede switch.

A popular object expires. Without this, every request that arrives in the next 200 ms is a MISS and every one of them goes to the origin. With it, one goes and the rest wait for the answer.

Off by default.

```
proxy_cache_use_stale error timeout
    updating http_500 http_502 http_503 http_504;
```

Serve stale rather than fail.

Your origin is down. You have a copy from four minutes ago. Which would your users rather have — a slightly old page, or a 502?

`updating` also means one request refreshes while everyone else keeps getting the stale copy instantly.

```
$ curl -i localhost:8082/ | grep X-Cache      →  X-Cache-Status: MISS
$ curl -i localhost:8082/ | grep X-Cache      →  X-Cache-Status: HIT
```

add_header X-Cache-Status $upstream_cache_status; — put this in every cache config you ever write.

location precedence has nothing to do with the order you wrote them in

#	form	rule	request	result
1	`location = /exact`	Exact match. Wins immediately, search stops.	`/exact`	1
2	`location ^~ /images/`	Longest prefix, and it suppresses the regex search.	`/images/a.php`	2
3	`location ~ \.php$`	Regex. FIRST match in file order, not longest.	`/other/a.php`	3
4	`location /`	Longest prefix, used only if no regex matched.	`/anything`	4

Make them predict all four answers before you press enter. Roughly half the room gets /images/a.php wrong — they expect the regex to win.

This exact config is on :8081 in the repo.

nginx has no .htaccess.

That is not a missing feature. It is the reason nginx does not stat() its way up the directory tree on every single request.

Why 500 prefixes are free and 500 regexes are not

```
n  = (len <= node->len) ? len : node->len;
rc = ngx_filename_cmp(uri, node->name, n);

if (rc != 0) {
    node = (rc < 0) ? node->left : node->right;   /* a TREE */
    continue;
}

if (len > node->len && node->inclusive) {
    node = node->tree;  uri += n;  len -= n;      /* descend */
}
```

- Prefixes: a ternary search tree, built at config-parse time, walked one URI segment at a time.
- Regexes: a plain array, walked in order, every request.

The data structure is the documentation. Once you have seen that prefixes are a tree and regexes are an array, the precedence rules stop being trivia you memorise and become the only rules those two structures could have.

src/http/ngx_http_core_module.c ngx_http_core_find_location() ~L1446 · ngx_http_core_find_static_location() ~L1542

Smooth weighted round robin, measured

```
peer->current_weight += peer->effective_weight;
total                += peer->effective_weight;

if (peer->effective_weight < peer->weight)
    peer->effective_weight++;        /* passive health check */

if (best == NULL || peer->current_weight > best->current_weight)
    best = peer;    ...
best->current_weight -= total;
```

weights 3 : 1

naive WRR

a a a b

nginx (smooth)

a b a a

Same ratio.

No bursts at the origin.

```
$ for i in $(seq 24); do curl -s "localhost:8082/x-$i-$RANDOM" -o /dev/null; done
$ grep -o 'upstream=[0-9.:]*' logs/access.log | sort | uniq -c
      6 upstream=127.0.0.1:8001            18 upstream=127.0.0.1:8004     <- exactly 3:1
```

ngx_http_upstream_round_robin.c ngx_http_upstream_get_peer() ~L811 · run it: 07-nginx, port 8082

Fallbacks: one line, and every framework's pretty URLs work

```
location / {
    try_files $uri $uri/ /index.html?$args;
}
```

“Is this a real file? A real directory? No? Then hand the whole URI to the app and let it route.”

The last entry is the only one that is never tested. It is the fallback, not a candidate.

Fallback across a network boundary

```
location /maybe/ {
    try_files $uri @app;
}
location @app {
    proxy_pass http://app;
}
```

A NAMED location: reachable only from try_files or error_page, never from a URL. Try disk first; if it is not there, ask the upstream.

WordPress, Rails, Django, every SPA router: all of them are this directive plus an application that reads REQUEST_URI. Once you see it, the magic evaporates.

ngx_http_try_files_module.c ngx_http_try_files_handler() ~L79 · demo: port 8084

Line numbers drift. Function names do not.

```
$ git clone --depth 1 https://github.com/nginx/nginx.git
$ cd nginx && git log -1 --oneline
  231a60ee nginx-1.31.5-RELEASE

$ grep -rn "^ngx_epoll_process_events" src/
```

nginx puts the return type on its own line.

So ^name(finds the definition and nothing else — not the declarations, not the call sites. One habit, and the whole tree becomes navigable.

Everything in the next hour gives you a function name first and a line number second. The line numbers are pinned to 1.31.5 and they will be wrong by the time you read this. The names will not be.

That is not a disclaimer. It is the skill: navigate a codebase by structure, not by coordinates.

The tree in two minutes

src/core/	nginx's own standard library. string, array, list, hash, rbtree, queue, palloc, buf, conf, cycle, connection. It has its own everything because in 1999 you could not portably rely on anyone else's.
src/event/	The loop. modules/ holds six ways to wait — epoll, kqueue, select, poll, devpoll, eventport. Same interface, chosen at ./configure time.
src/http/	The HTTP state machine, the phase engine, the upstream framework. modules/ is every directive you have ever typed.
src/os/unix/	The syscall layer. sendfile per OS, process spawning, shared memory, atomics.
src/stream/	The same architecture again, for raw TCP and UDP. nginx as an L4 load balancer.
src/mail/	And a third time, for IMAP, POP3 and SMTP proxying.

Three protocol trees, one core. When you have read src/http/ you can read src/stream/ in an afternoon — and that repetition is the argument for the architecture. Say it out loud in class.

Two files to read before any of the code

src/core/ngx_queue.h

~50 lines of macros. An intrusive doubly-linked list: the link pointers live inside the node, so putting something in a list costs no allocation and getting from a link back to the data is a pointer subtraction.

The cache LRU, the posted-events queues and half of src/core/ ride on it.

src/core/ngx_palloc.h

The memory pool. nginx does not free per-request memory. It allocates from a pool attached to the request and destroys the whole pool at the end.

No refcounting. No leak class. No free() on the hot path — and almost no cleanup branches anywhere in the codebase.

Read these two and a large fraction of nginx stops looking like C and starts looking like a language with a runtime. Skip them and every function has three constructs in it you do not recognise.

Ten stops

what	file	function	in the repo
Process model	os/unix/ngx_process_cycle.c	ngx_start_worker_processes	01-fork
The loop	event/ngx_event.c	ngx_process_events_and_timers	04-epoll
Accept + herd	event/ngx_event_accept.c	ngx_event_accept	04-epoll
epoll + stale events	event/modules/ngx_epoll_module.c	ngx_epoll_process_events	04-epoll
The parser	http/ngx_http_parse.c	ngx_http_parse_request_line	03-select
Phases	http/ngx_http_core_module.c	ngx_http_core_run_phases	—
Routing	http/ngx_http_core_module.c	ngx_http_core_find_static_location	07-nginx :8081
sendfile	os/unix/ngx_linux_sendfile_chain.c	ngx_linux_sendfile	05-sendfile
Cache LRU	http/ngx_http_file_cache.c	ngx_http_file_cache_expire	07-nginx :8082
FastCGI records	http/modules/ngx_http_fastcgi_module.c	ngx_http_fastcgi_process_record	06-fastcgi

Full tour with line numbers, code excerpts and Socratic questions: NGINX-TOUR.md in the repo.

The HTTP parser is a resumable state machine, not a string function

```
enum {
    sw_start = 0, sw_method, sw_spaces_before_uri, sw_schema,
    sw_schema_slash, sw_schema_slash_slash, sw_host_start,
    sw_host, sw_port, sw_after_slash_in_uri, sw_check_uri,
    sw_uri, sw_http_09, sw_http_H, sw_http_HT, ...
} state;

state = r->state;                  /* <- resume where we stopped */
for (p = b->pos; p < b->last; p++) {
    ch = *p;
    switch (state) { ... }
```

- **27 states for the request line alone.**
- No regex. No strtok. No substring. No allocation.
- One byte at a time, and it can stop between any two.

Because the request line may arrive in three TCP segments. r->state survives between calls, so the parser picks up mid-method if it has to.

This is what non-blocking costs you at the source level: every parser has to be resumable, because you can be interrupted anywhere.

ngx_http_parse.c ngx_http_parse_request_line() ~L108 · ..._header_line() ~L871 · ngx_http_request.c ..._alloc_large_header_buffer() ~L1657

Eleven phases, four lines, and `return` means “suspended”

POST_READ → SERVER_REWRITE → FIND_CONFIG → REWRITE → POST_REWRITE → PREACCESS → ACCESS → POST_ACCESS → PRECONTENT → CONTENT → LOG

```
while (ph[r->phase_handler].checker) {
    rc = ph[r->phase_handler].checker(r, &ph[r->phase_handler]);
    if (rc == NGX_OK) { return; }    /* NOT done. SUSPENDED. */
}
```

A coroutine, hand-rolled, in C, in 1999.

The request resumes in a later loop iteration when its event fires, re-entering this same while at the phase it left off.

```
$ grep -rn "NGX_HTTP_.*_PHASE\].handlers" src/http/modules/
  # the map of which module runs when. one-line answer to "why did my directive not fire?"
```

limit_req in PREACCESS · auth_basic in ACCESS · try_files in PRECONTENT · proxy_pass, fastcgi_pass and static in CONTENT.

ngx_http_core_module.c ngx_http_core_run_phases() ~L894 · ngx_http_core_module.h ngx_http_phases enum ~L111

FastCGI: framing is length or delimiter. This one uses both.



8-byte record header → contentData[contentLength] → padding

```
>> BEGIN_REQUEST
   01 01 00 01 00 08 00 00   <- the first 8 bytes of every
                                FastCGI connection on earth
>> PARAMS    <- EMPTY = end of stream
   01 04 00 01 00 00 00 00
```

The empty record IS the delimiter.

An empty PARAMS record means “no more environment variables”. An empty STDIN means “no more body”. There is no other end-of-stream signal. Forget it and the responder waits forever.

A length inside every header, and an empty record between streams. Session 2's closing idea, at two levels, in one twenty-page spec.

ngx_http_fastcgi_module.c ngx_http_fastcgi_process_record() ~L2742 · ..._create_request() ~L858 · 06-fastcgi/

The ladder, in code you can hold in your head

01-fork	**fork() per client**	Apache 1.3, 1995	ps shows one process per in-flight request
02-thread	**pthread per client**	Apache 2.0 worker, 1999	prints its own 8 MB stack size at startup
03-select	**one thread, select()**	…and the 1024 wall	dies at fd 1024, and ulimit does not help
04-epoll	**one thread, epoll()**	nginx, 2004	5,000 connections, nothing happens
05-sendfile	**read vs mmap vs sendfile**	the copies, counted	2048 syscalls become 1
06-fastcgi	**the wire protocol, both ends**	CGI without the fork	nginx ⟶ your own 200-line responder
07-nginx	**one nginx.conf, five demos**	all of it, for real	routing, cache, weights, fallbacks

About 1,200 lines total. nginx is 200,000, and the difference is every error path we ignored. That is the honest reason nginx is big — not complexity for its own sake.

CLOSE

Homework

1 **Break select, then fix it**

flood 03-select at 1,100. Find the line where it gives up. Delete the FD_SETSIZE guard, run it again, capture the real crash. Then explain in two sentences why ulimit -n 65536 does not help — and run the same flood at 5,000 against 04-epoll.

2 **Find the sendfile decision**

Start from `sendfile on;` in a config and get to sendfile(2) in ngx_linux_sendfile_chain.c, writing down every function you pass through. Then add `gzip on;`, re-run, and show with strace that sendfile is gone. Which filter took it away?

3 **Speak FastCGI to something real**

Point 06-fastcgi/fcgi_client at a real php-fpm and get a PHP page back. Then tcpdump loopback while nginx does the same request and find FCGI_BEGIN_REQUEST by hand. Diff nginx's PARAMS against ours — what does it send that we don't, and why?

4 **Make round robin lie to you**

Put a 10-second sleep in one upstream, leave the weights at 3:1, send 100 requests. Compare p99 under round robin and under least_conn. Compute the concurrency Little's Law predicts, then check it against `ss -tn | wc -l`.

5 **Read one module end to end**

ngx_http_try_files_module.c, 390 lines. Write one page on what happens to `try_files $uri $uri/ /index.html?$args` for a URI that does not exist — every stat, every allocation, where the URI gets rewritten. Then explain why the last argument is never tested.

CLOSE

Four ideas to leave with

1

Ask the other question

Apache spent a decade making the per-connection worker cheaper. nginx asked why there was one. When a whole industry is optimising the same thing, the opportunity is usually one level up.

2

Check whether the constraint still exists

Thread-per-connection died because a thread cost 8 MB and a context switch. Goroutines and virtual threads brought it back when those numbers changed. Know which of your rules are laws and which are 2004.

3

The data structure is the documentation

Prefixes are a tree, so longest wins. Regexes are an array, so first wins. The precedence rules are not trivia — they are the only rules those structures could have.

4

Amortise the setup

Prefork, pre-thread, servlets, connection pools, FastCGI, upstream keepalive. Six names for one move: take the expensive setup off the request path and pay it once.

NEXT SESSION

Where did all the state go?

We put a reverse proxy in front of everything and made the app servers interchangeable. Sessions, sticky routing, TLS termination, shared caches, and what “stateless” actually costs.

github.com/nginx/nginx · nginx 1.31.5 · repo: nginx-from-scratch · tour: NGINX-TOUR.md